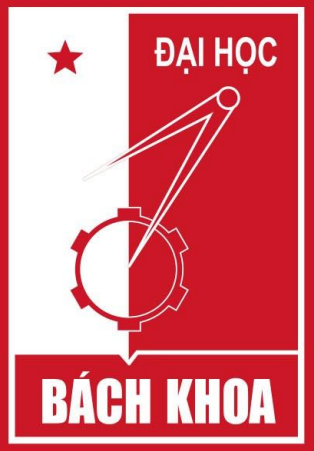




ĐẠI HỌC BÁCH KHOA HÀ NỘI
VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

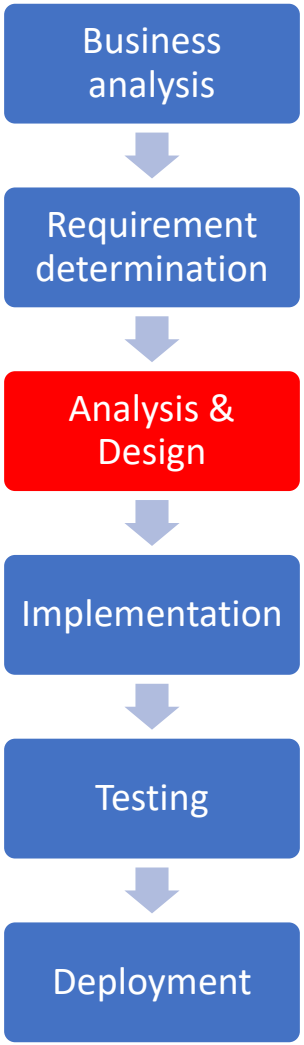
Lecture 3
Analysis & Design

Content

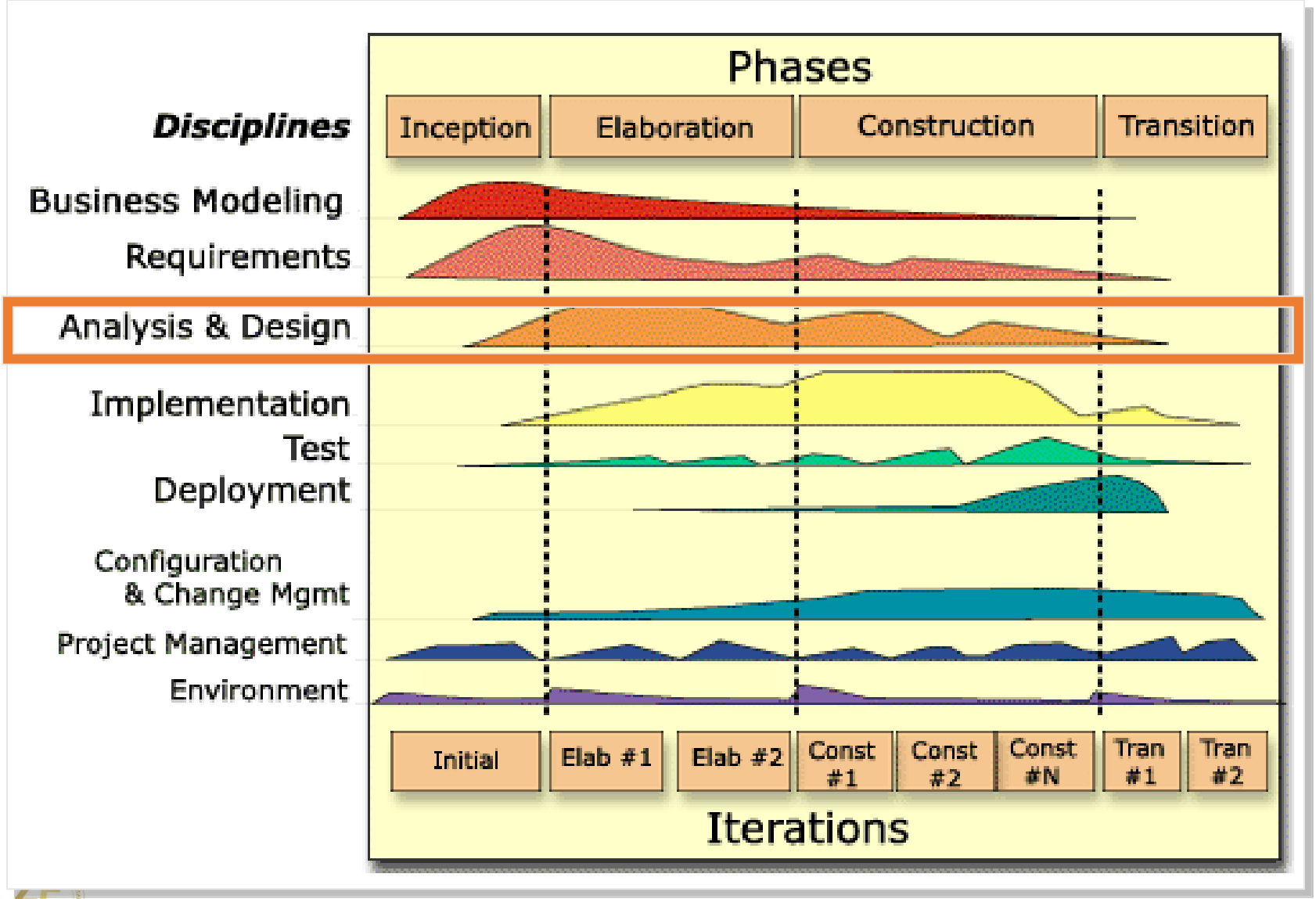
1. Overview
2. Modeling the system
    - 2.1. Structural modeling
        - ECB pattern
    - 2.2. Behavioral modeling
        - Interaction behavioral modeling
        - State-based behavioral modeling

1. Overview

Analysis & Design



A&D in RUP


Phases
Disciplines
Inception
Elaboration
Construction
Transition
Business Modeling
Requirements
Analysis & Design
Implementation
Test
Deployment
Configuration
& Change Mgmt
Project Management
Environment
Initial
Elab #1
Elab #2
Const #1
Const #2
Const #N
Tran #1
Tran #2
Iterations

Purposes

A&D aims to create a system model that:

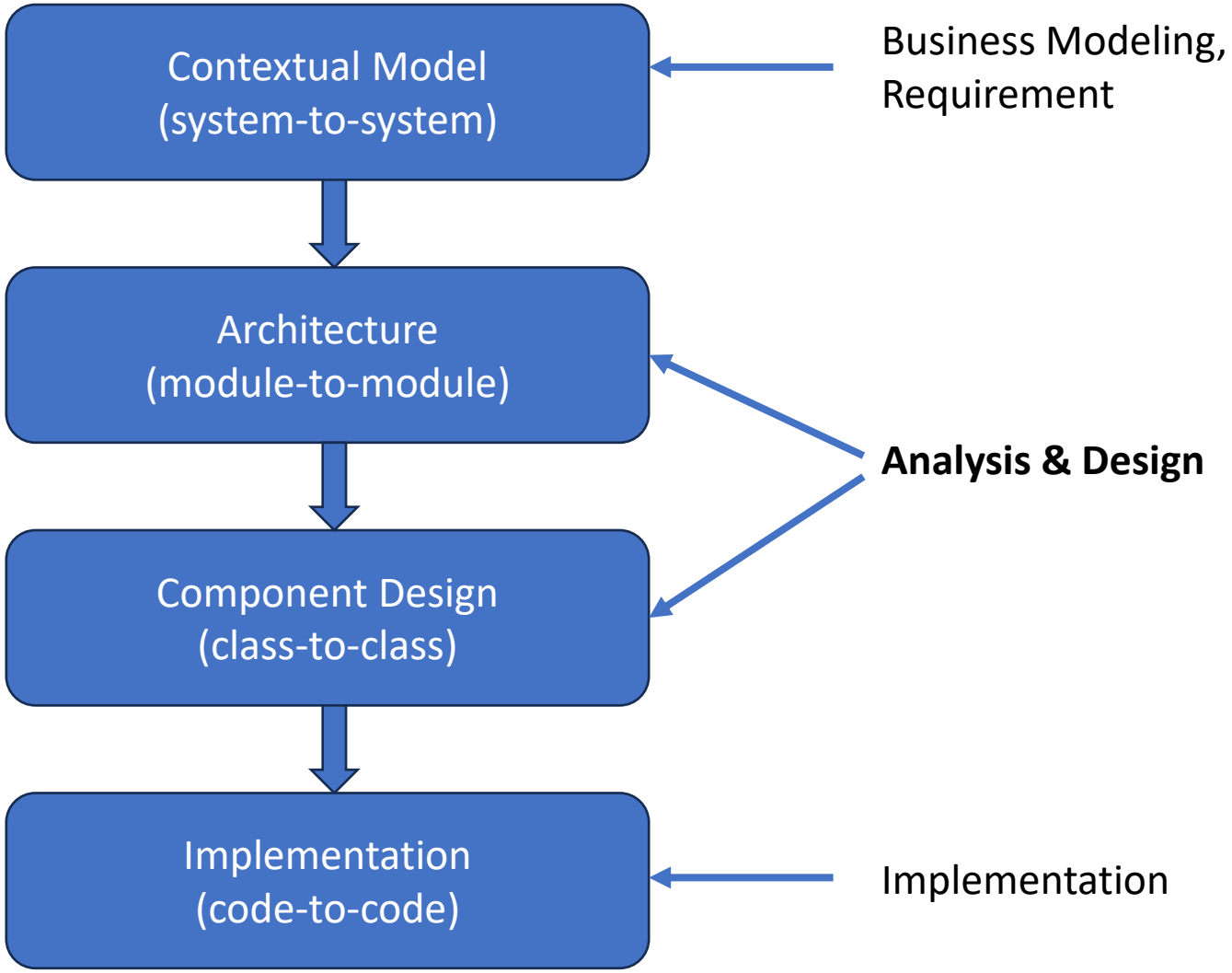
- aligns with the system requirements (functional & non-functional)
- is feasible to implement



System model

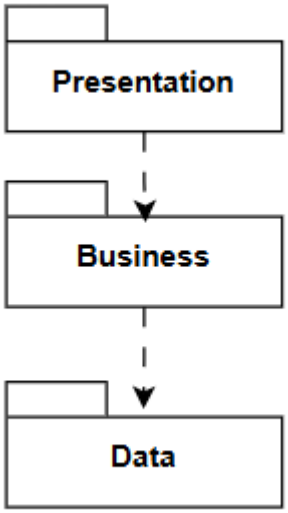
- Structural models
  - describes static structure of the system
  - UML:
    - Logical structure →class diagram
    - Subsystem structure → component diagram
    - Physical infrastructure → deployment diagram
    - Organizational structure → package diagram
- Behavioral models
  - describes dynamic aspects of the system
  - UML
    - Interactions → sequence diagram, communication diagram
    - Flow of logic → activity diagram
    - Behavior in the lifecycle of an object → state machine diagram

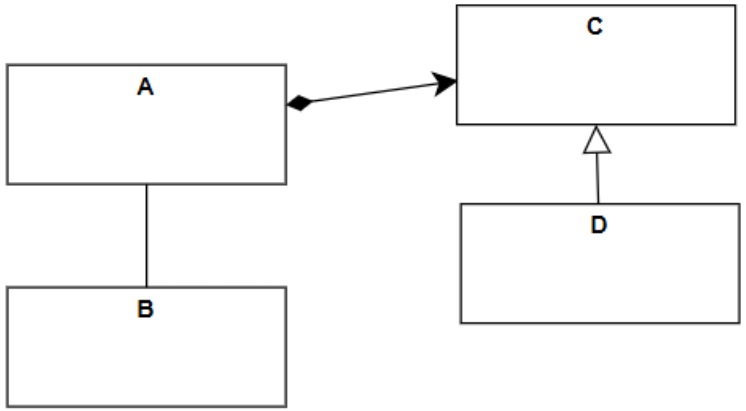
Levels of abstraction



Architecture vs. Component Design

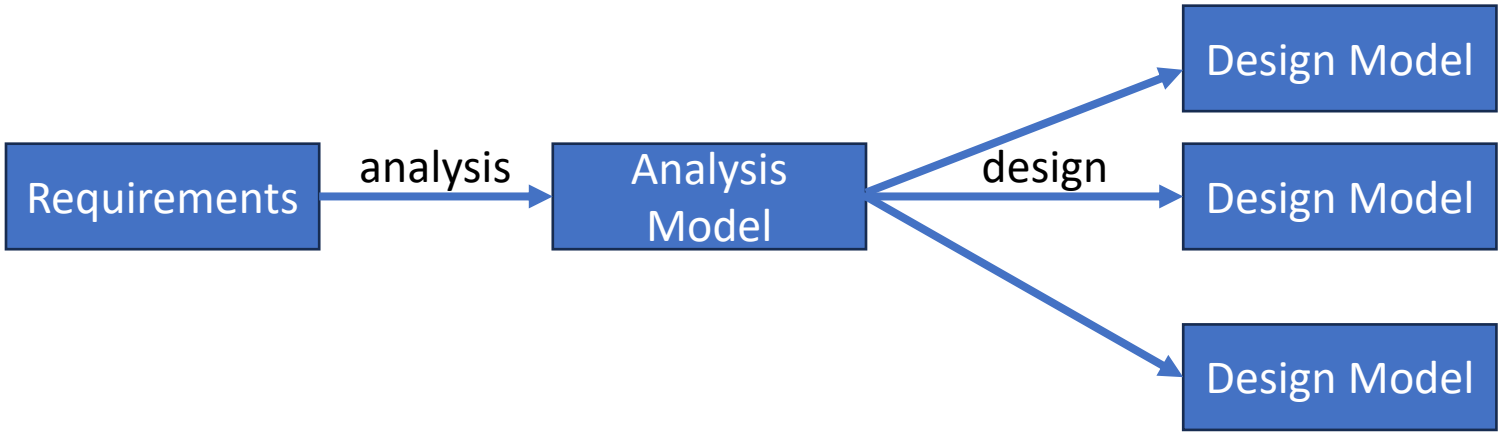
- Architecture
  - High level abstraction
  - Expensive to change
  - System topology, infrastructure, code structure, component interaction
- Component Design
  - Low level abstraction/Details
  - Costs for refactoring are acceptable
  - Class structure, object interaction, flow of logics





Analysis vs. Design

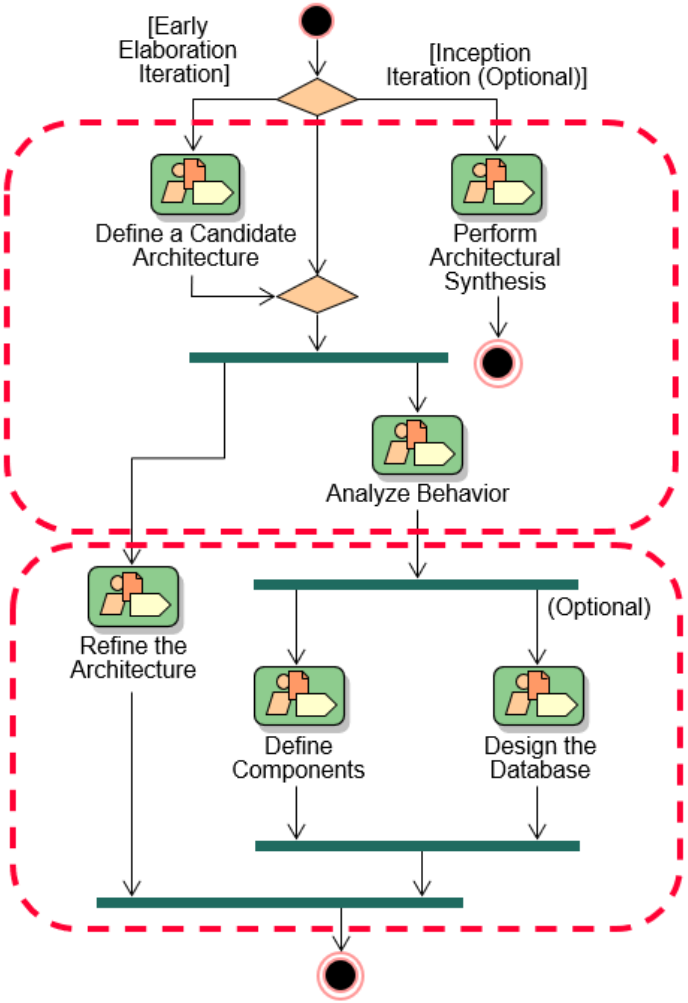
- Analysis
  - understand the problem
  - focus on functional requirements
  - High level structure
  - Idealized & small model
- Design
  - understand the solution
  - focus on non-functional requirements
  - Component design & class details
  - Technical & large model



A&D Process in RUP

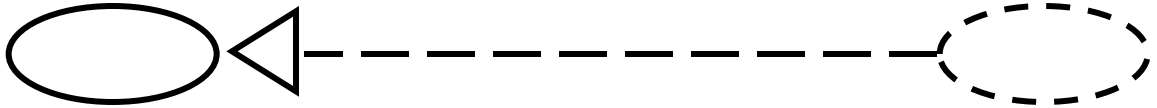
ANALYSIS

DESIGN

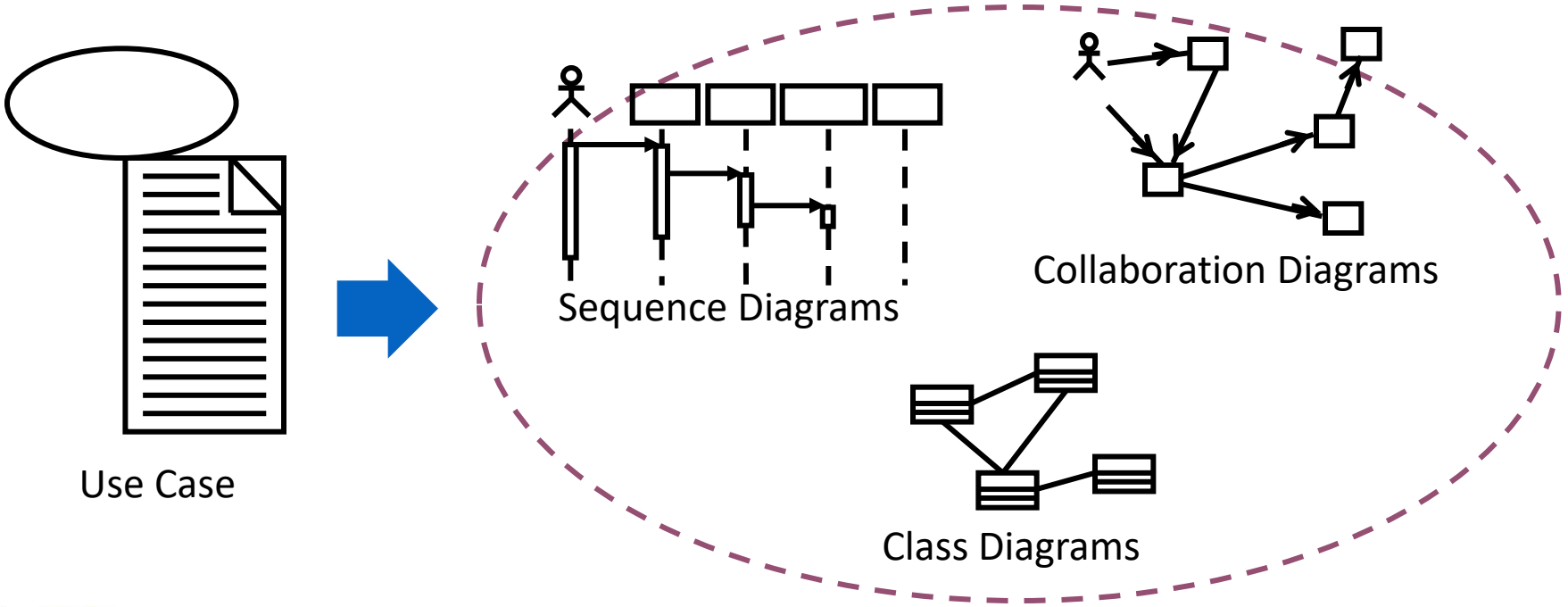


Use Case Realization

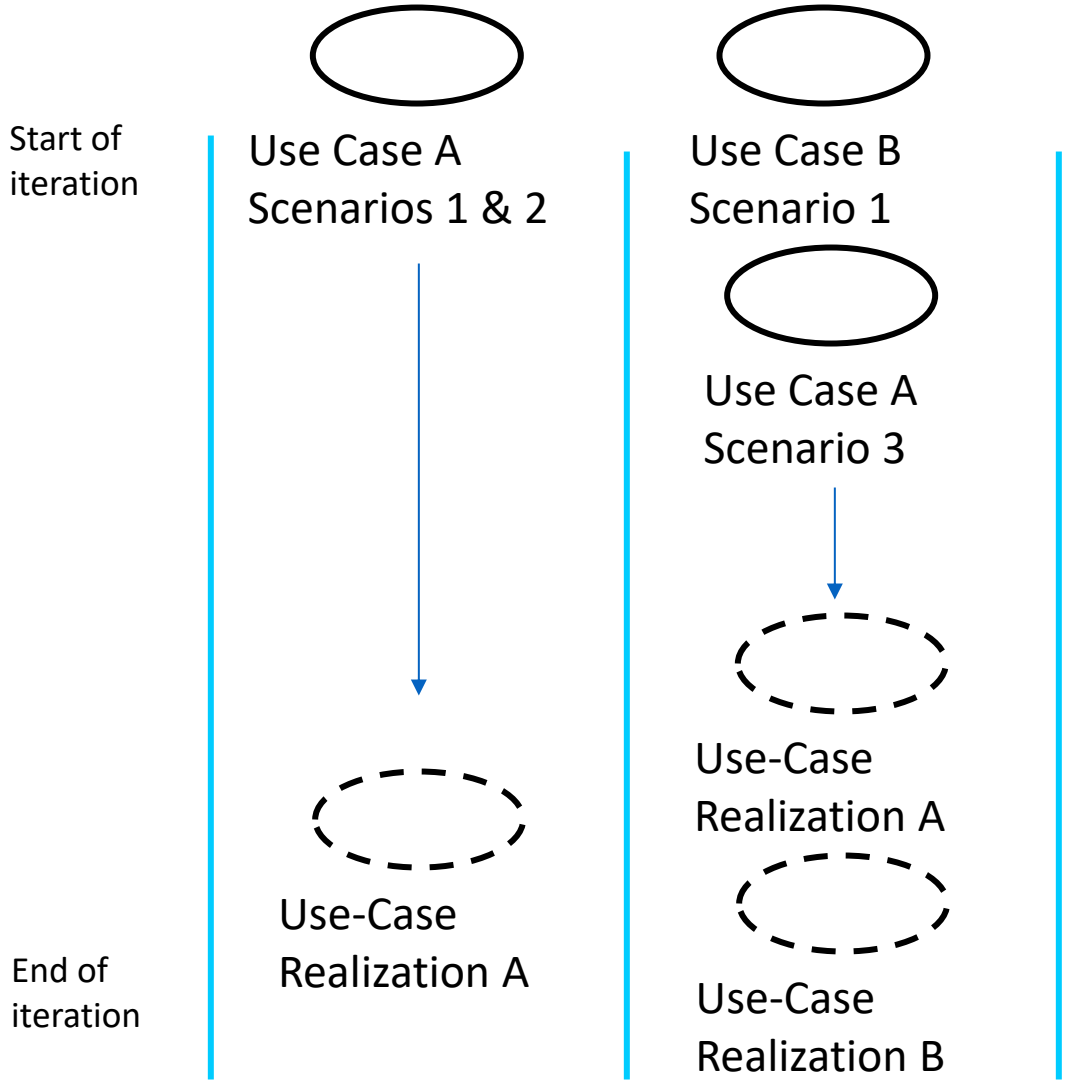
Use-Case Model ***Design Model***



Use Case Use-Case Realization



A&D in an Iterative Process



2. Structural Modeling

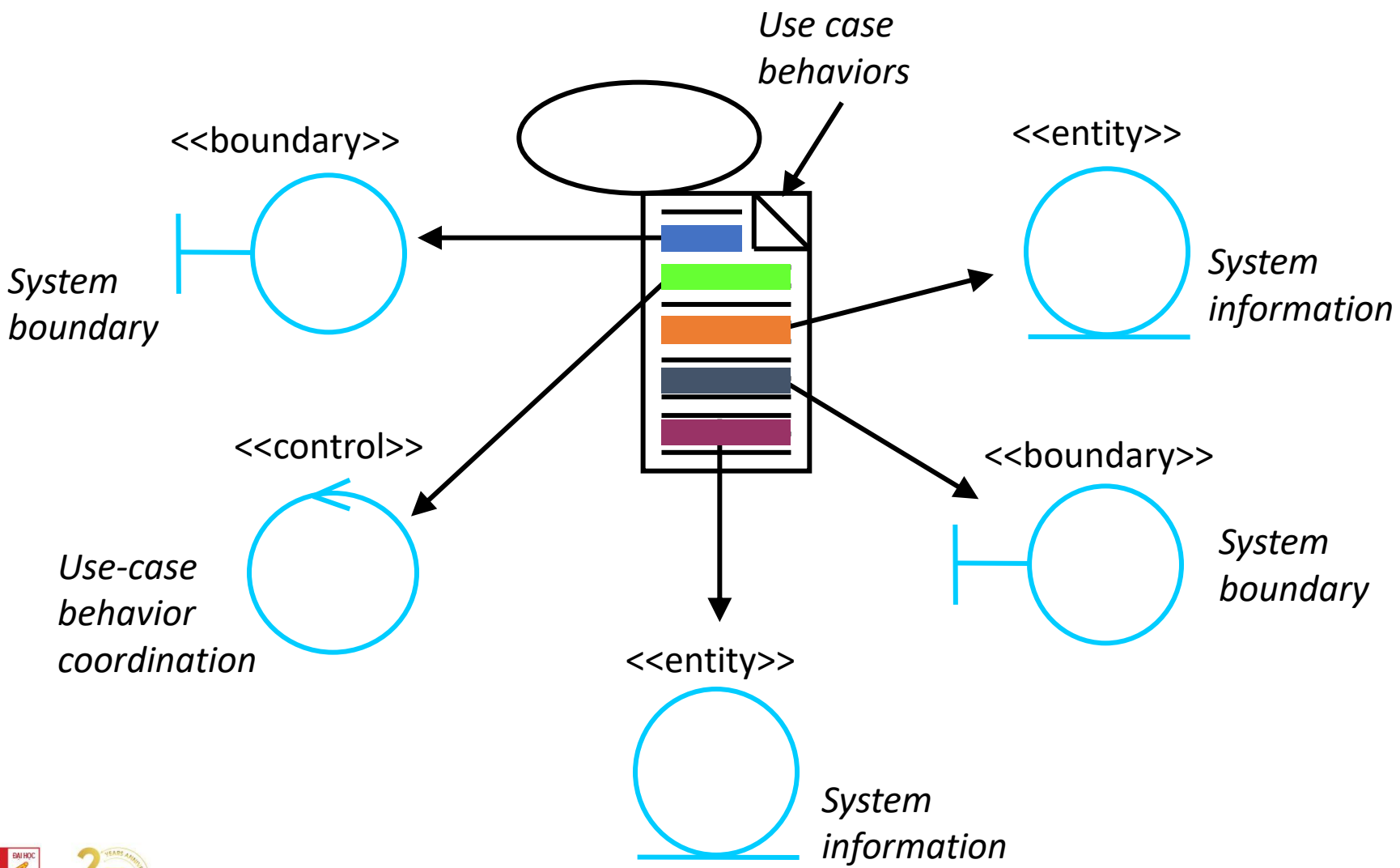
Structural Modeling

- Structural modeling is the technique that desribes the static structure of the system.
  - UML:
    - Logical structure →class diagram
    - Subsystem structure → component diagram
    - Physical infrastructure → deployment diagram
    - Organizational structure → package diagram
- Structural modeling is applied in both analysis & design phases
  - Analysis: architectural framework, analysis classes (ECB pattern) ← WE FOCUS HERE
  - Design: architecture, component design, class design, infrastructure

ECB Pattern

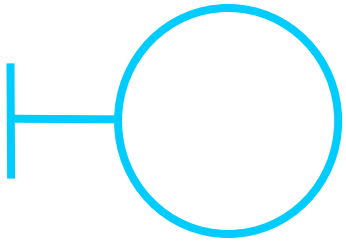
- Entity-Control-Boundary (EBC) pattern is an analysis technique that is often applied in use case analysis to identify analysis (conceptual) classes, their responsibilities, their collaboration that fulfill the requirements of the use case.
- For each use case
  - Identify analysis classes
    - Boundary classes
    - Control classes
    - Entity classes
  - Distribute use case behavior to classes
  - Describe the responsibility, attributes, associations for each class
- Unify analysis classes to create analysis structural model of the system
  - In Iterative & Incremental process: unify all analysis classes from all use case realizations that have been done up to the current iteration

Analysis Classes



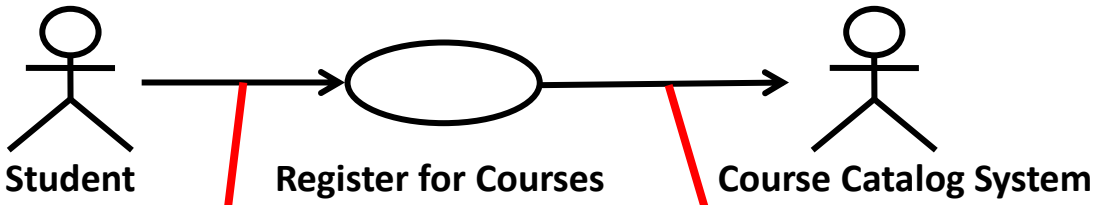
Boundary Class

- Intermediates between the interface and something outside the system
  - Model the interaction between the system & its environment (actors)
- Several Types
  - User interface classes
  - System interface classes
  - Device interface classes
- *One boundary class per actor/use-case pair*

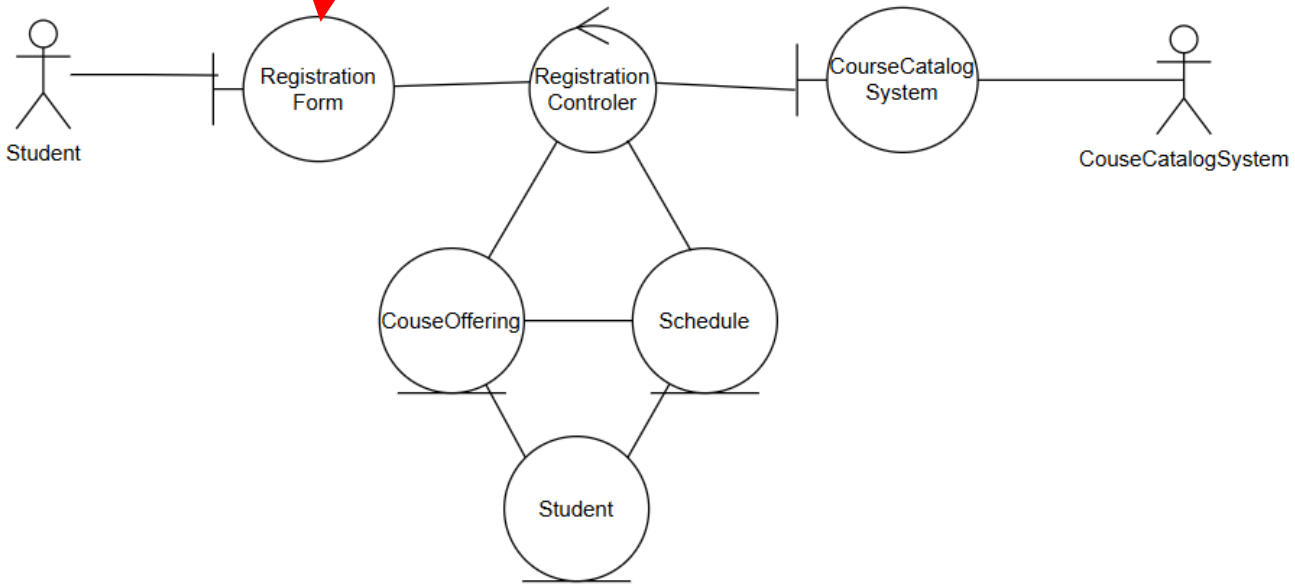


Boundary Class: Example

Use case model



Analysis model

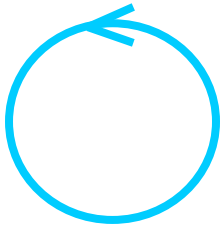


Boundary Class: Guideline

- User Interface Classes
  - Concentrate on what information is presented to the user
  - Do NOT concentrate on the UI details
- System and Device Interface Classes
  - Concentrate on what protocols must be defined
  - Do NOT concentrate on how the protocols will be implemented

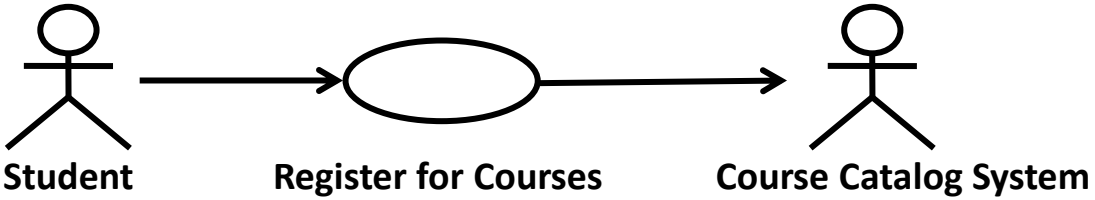
Control Class

- Use case behavior coordinator
  - More complex use cases generally require one or more control classes

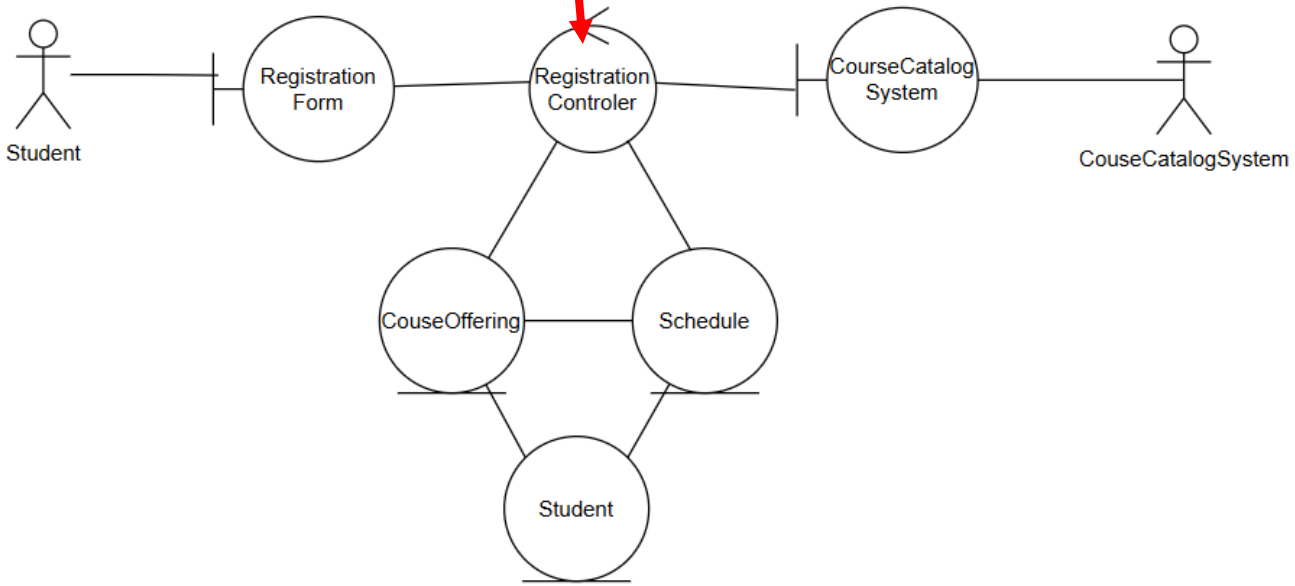


Control Class: Example

Use case model



Analysis model



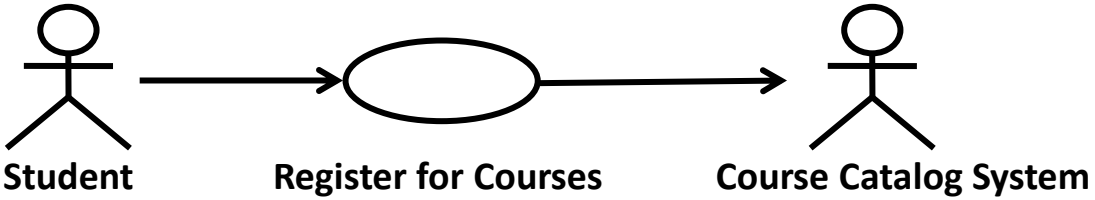
Entity Class

- Store & manage information of the system

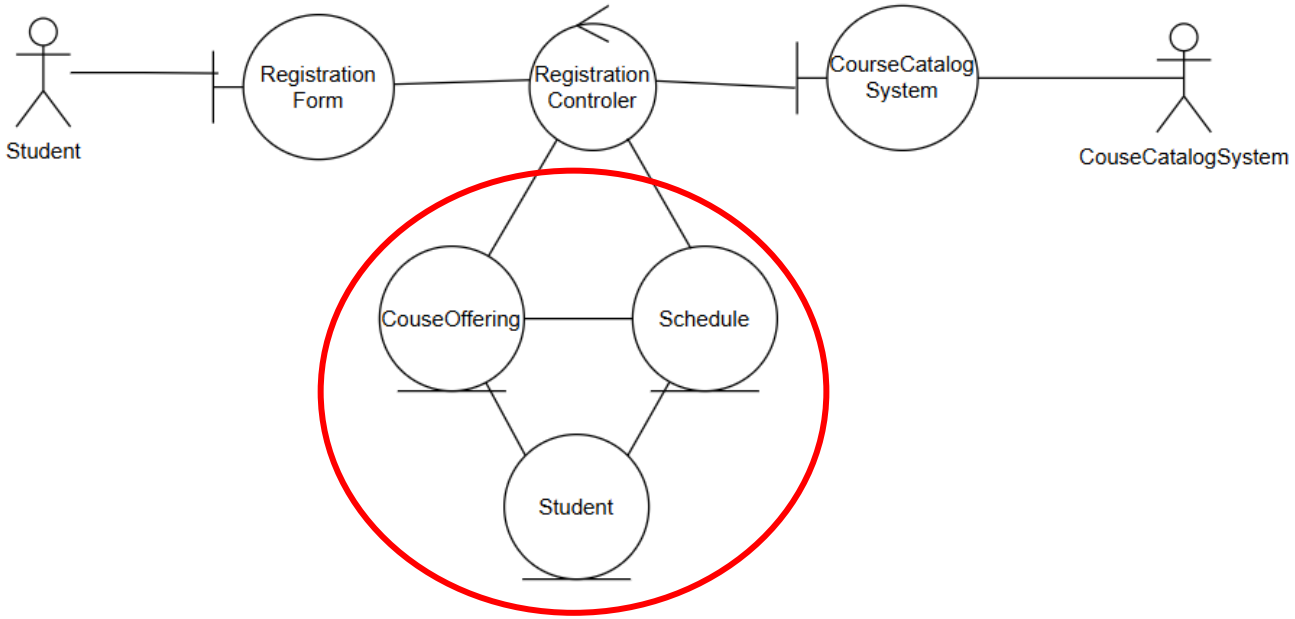


Entity Class: Example

Use case model



Analysis model



Entity Class: Guideline

- Use use-case flow of events as input
- Filtering nouns approach
  - Underline noun clauses in the use-case flow of events
  - Remove redundant candidates
  - Remove vague candidates
  - Remove actors (out of scope)
  - Remove implementation constructs
  - Remove attributes (save for later)
  - Remove operations

Operation, Attribute, and Association

- Operations
  - In an analysis class, operations represent the responsibilities of the class to the behavior of the system that described in the event flow.
  - Boundary class:
    - Providing information to actors
    - Performing requests from actors
  - Control class:
    - Responsibilities that are distributed from Boundary classes or other Control class
  - Entity class:
    - Information query or manipulation



Operation, Attribute, and Association

- Attributes
  - Boundary class:
    - Inputs from actors
    - Output to actors
  - Control class:
    - Intermediate states of the coordination process
  - Entity class:
    - Properties of the entity



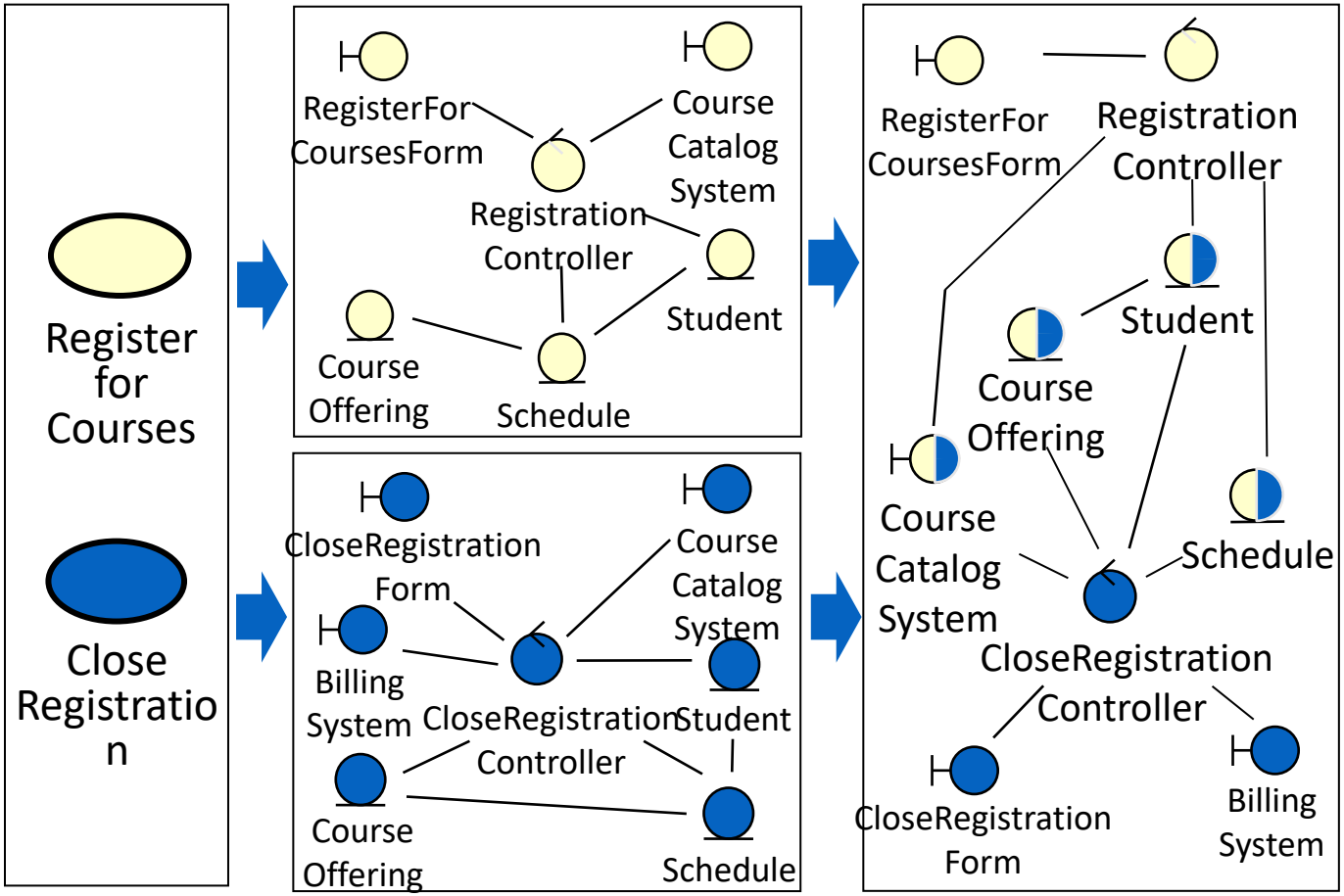
Operation, Attribute, and Association

- Associations
  - Boundary-to-Control
    - The responsibilities are dispatched from a Boundary class to a Control class or inverse
  - Control-to-Control:
    - The responsibilities are dispatched among Control classes
  - Control-to-Entity
    - The responsibilities are dispatched from a Control class to an Entity class
  - Avoid connecting a Boundary class to an Entity class



Unify Analysis Classes

- For each use case, an analysis class model of the system is created. At the end of the analysis phase, a unified class model of the system will be needed.



3. Behavioral Modeling

Behavior of the system

Behaviors of the system can be modeled in the following aspects:

- Interaction
  - UML Sequence diagram
  - UML Communication diagram
- Flow of logics
  - UML Activity diagram
- Dynamics behaviors of an object along its lifecycle (state-based behavioral model)
  - UML State machine diagram

Interaction model

- An **Interaction Model** is a dynamic representation that focuses entirely on how different elements of a system communicate and exchange data with one another to achieve a specific behavior.
  - Actor-to-System interaction
    - Sequence Diagram that models Actor-to-System interaction is called SSD (System Sequence Diagram)
  - Component-to-Component interaction (architecture)
  - Object-to-Object interaction (analysis model, detailed design model)
- In OO approach, the only way that one object communicates with another is **message passing.**

  → Interaction model represents the flows of message passing among objects
    - Sequence Diagram
    - Communication Diagram

Message

- Message is the fundamental unit of communication. It represents the information, data, or command sent from one sender to a receiver to trigger an action, share data, or request a response.
- Types of messages
  - Call message: request to do something at the receiver
  - Return message: respond from receiver to sender
  - Send message: send a signal/data to receiver
  - Create message: create an object
  - Destroy message: destroy an object
  - Reflexive message: a message an object send to itself

Synchronous Message & Asyncronous Message

- Synchronous message: the sender transmits a message to the receiver and blocks its own execution threads, waiting idling until the receiver processes the request and returns a response.



- Asynchronous message: the sender transmits a message to the receiver and immediately resumes its own execution without waiting for the receiver to process it.
  - If the sender need a respond, an explicit return message is required.



Lost Message & Found Message

- Lost message: a message that is sent by a known component, but the receiver is either unknown, outside the scope of the diagram, or the message never reaches its destination due to a network or system failure.

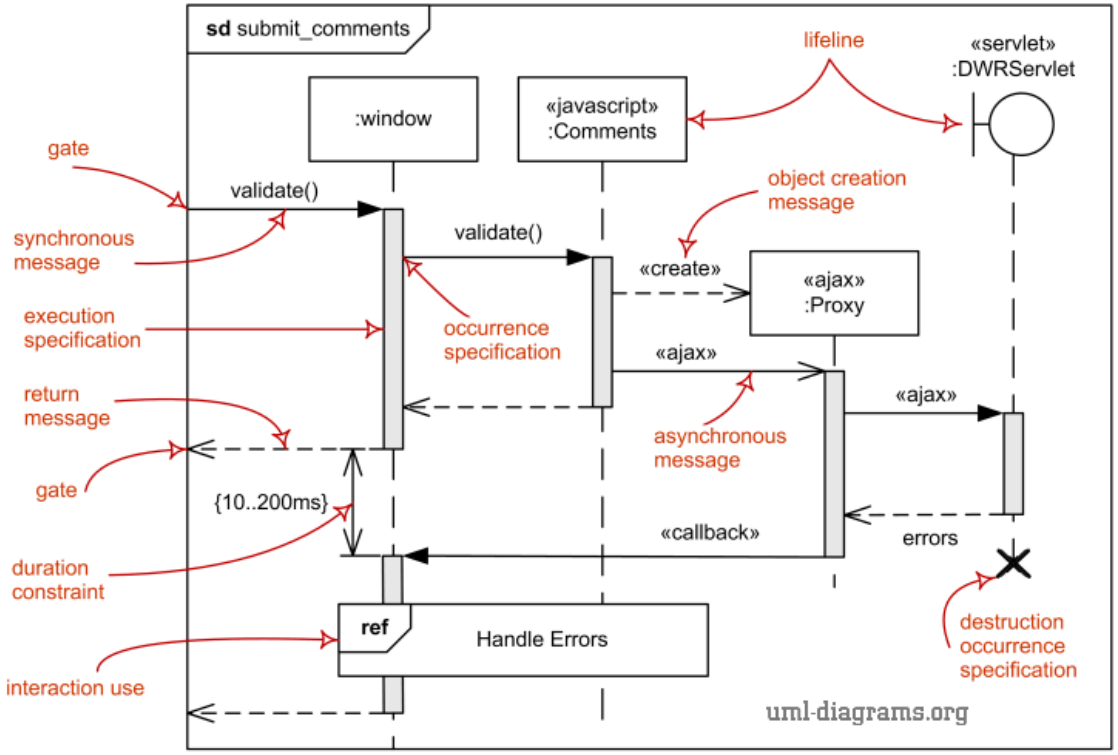


- Found message: a message that arrives at a known component, but the sender is completely unknown, anonymous, or outside the boundary of what you are currently modeling.



Sequence Diagram

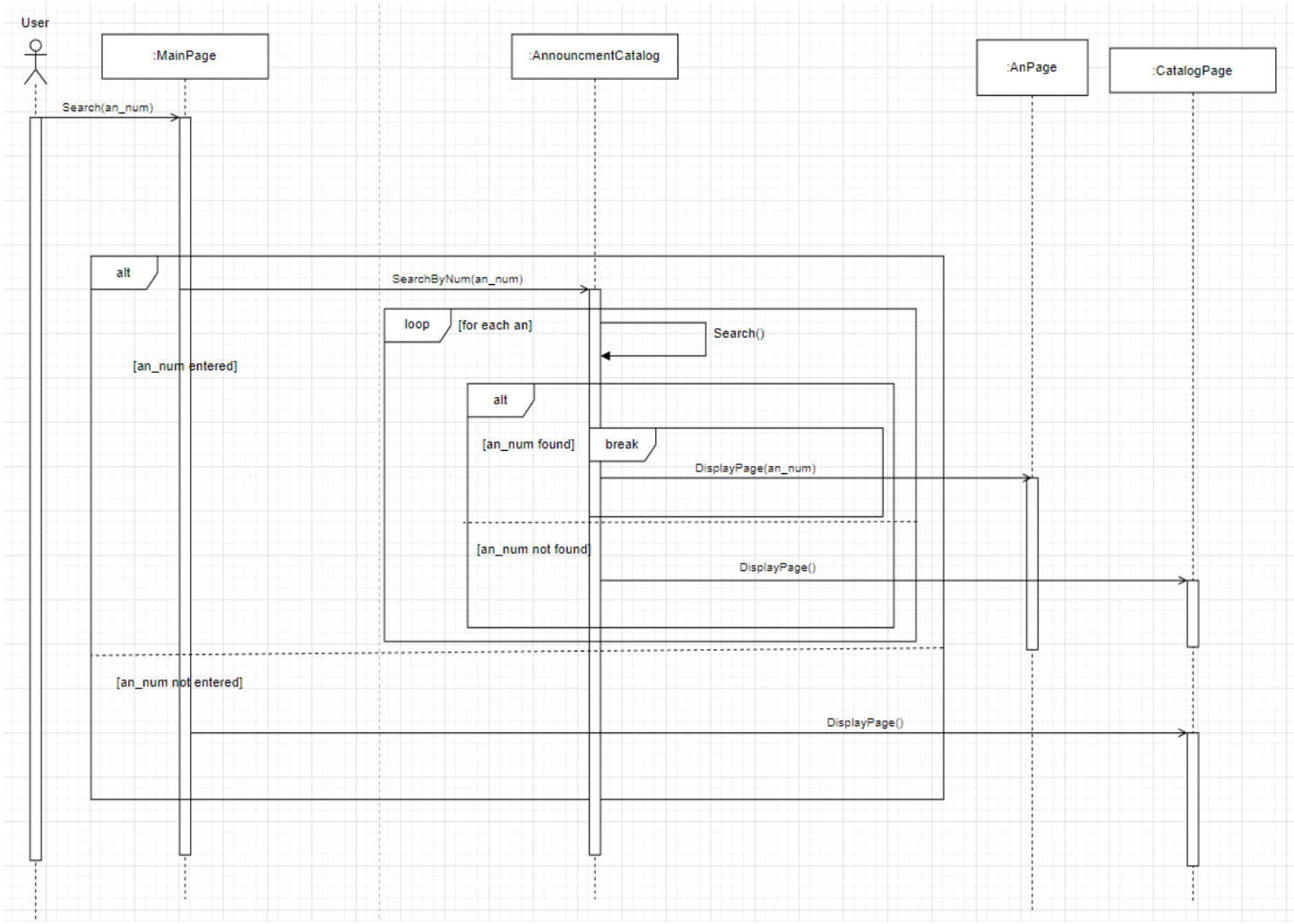
- Sequence diagram is a dynamic diagram that shows how objects, components, or microservices interact with one another **in a specific chronological order** to complete a business workflow.



Elements of Sequence Diagram

- Lifelines (Participants): represents an individual participant in the interaction—this can be an object, a user actor, a frontend client, a backend microservice, or a database. Time flows from **top to bottom** along these dashed lines.
- Activation Bars (Execution Specifications): They indicate that the component is currently active, running a process, or waiting for a response from another system.
- Messages
- Combined Fragments: manage logic conditions, mimicking programming syntax:
  - alt (Alternative): Represents an if/else condition.
  - opt (Optional): A step that only executes if a specific condition is met.
  - loop: Repeats a series of messages multiple times.

Combined Fragments: example



Message specification

```
[sequence number][Guard condition/iteration] : name[(parameters)]
```

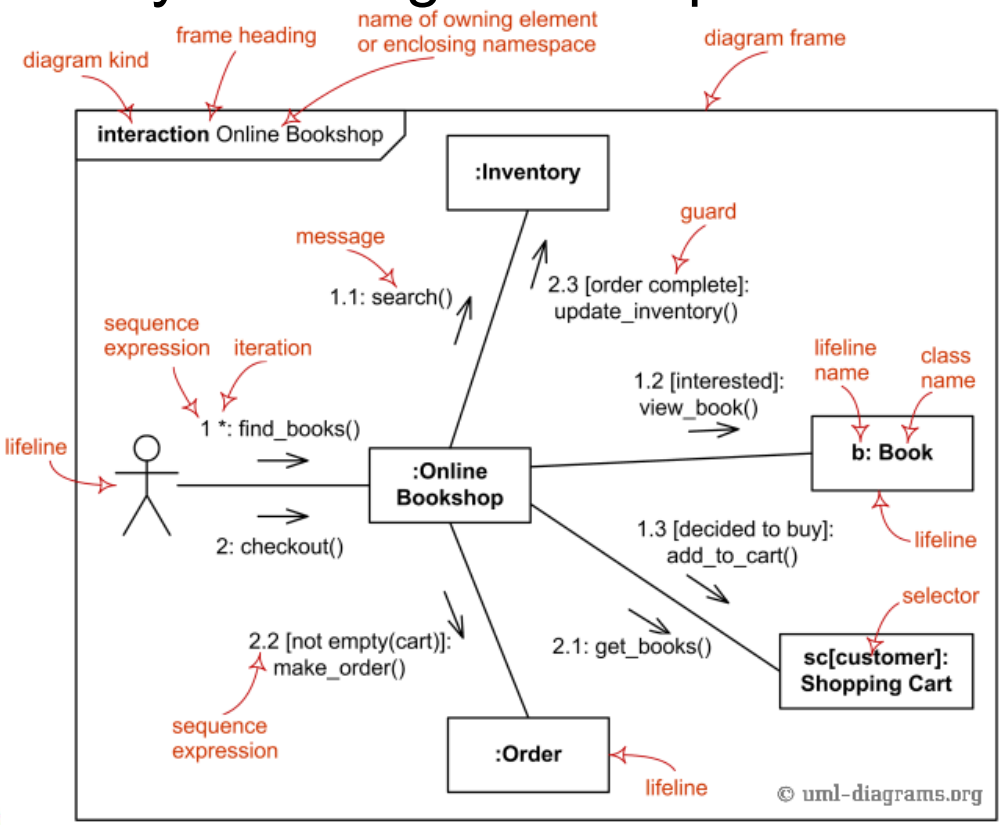
- Sequence number: specifying the order of the mesage
  - Can be nested
  - [Number]: sequential
  - [Alphabet]: parallel
  - Ex. “1.A”
- Guard condition: message can only be sent if the condition is satisfied.
  - Ex. [Out of stock]: display error message()
- Iteration expression:
  - Ex. *[for each item in the cart]: doSmt()

UC Analysis: Modeling the interaction with sequence diagram

- Analyzing the use case specification
  - Basic flow (happy path)
  - Alternative/Exception flows
- Identifying objects
  - Actors
  - Boundary
  - Control
  - Entity
- Drawing object lifelines from left to right: primary actor, boundary object, control object, entity object, secondary actors
  - Lowering object lifelines of objects which are created during the collaboration.
- Drawing messages from top to bottom according to the order of events and the responsibility distribution (outside-in).
- Specifying the messages: parameters, sequence number, guard condition
- Highlighting the activation bars

Communication Diagram

- Communication diagram organizes interactions around the **architectural relationships** between objects. It focuses on the structural network of the system, showing exactly which components are linked together and what messages they exchange to complete a specific task.

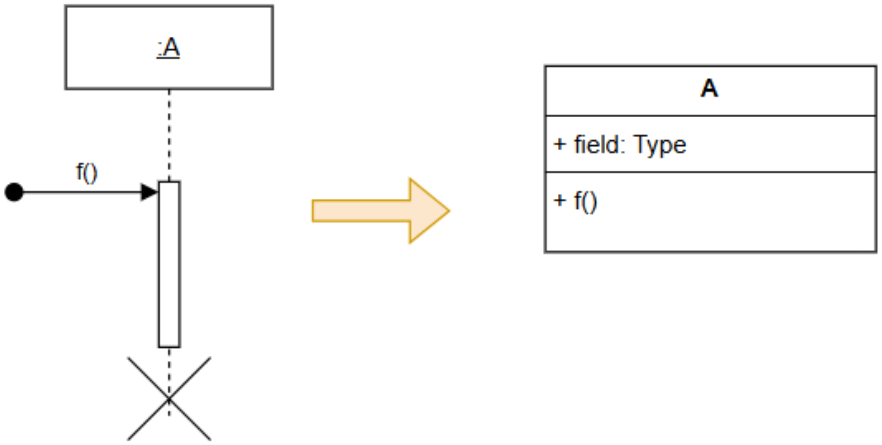


Elements of Communication Diagram

- Objects (participants)
- Links (connections): logical or physical path of the communication between objects
- Messages with sequence expressions: draw parallel to the link. Sequence numbers are crucial to know the execution order.

Invalidate the models

- Add and remove classes
  - Classes whose objects received messages (take responsibilities) should appear in the class diagram.
  - Classes without any behaviors (do not receive any message) should be eliminated from class diagram.
- Add and modify operations
  - An object of a class A receives a call message f → f would be an operation of A.



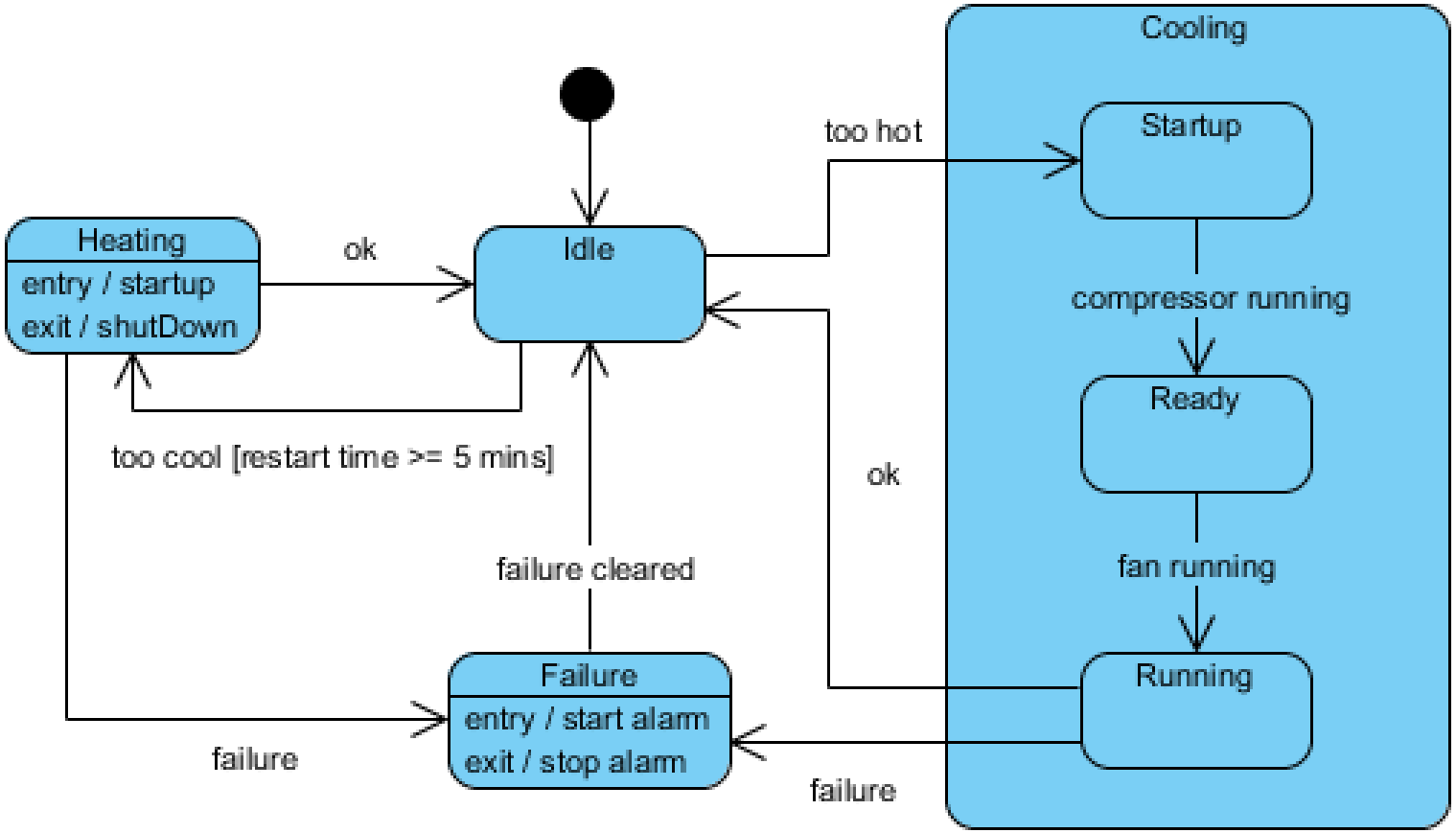
Invalidate the models

- Add and modify attributes of classes & parameters of messages
- Add and modify relationship among classes
  - Links between objects can be generalized to:
    - Association between classes
    - Dependency between classes
      - Global dependency
      - Local dependency
      - Parameter dependency

State-base behavioral model

- A **State-Based Behavioral Model** is a dynamic model that describes the behavior of a system or a single object by examining its lifecycle.
  - Most objects (passive objects) react uniformly to the incoming events
  - Some objects (active objects) react differently to the incoming events depending on its state in the lifecycle
- State machine diagram
  - Represents the states of an object along its lifecycle,the transitions among states
  - Describes the behaviors of objects at a specific state or on a transition

State Machine Diagram



Elements of State Machine Diagram

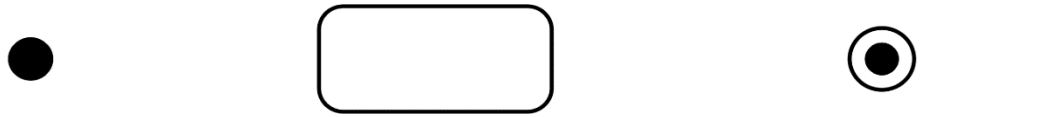
- Event
- State
- Transition

Event

- In State Machine Diagram, event is something happens to the system/object that affects the behavior of the system/object.
- Types of events
  - Message-based event:
    - Call event: object receives a call message
    - Signal event: object receives a send message
  - Non-message-based event
    - Time event:
      - Eg. At 9AM, After 10 seconds
    - Change event
      - Eg. Temperature > 100

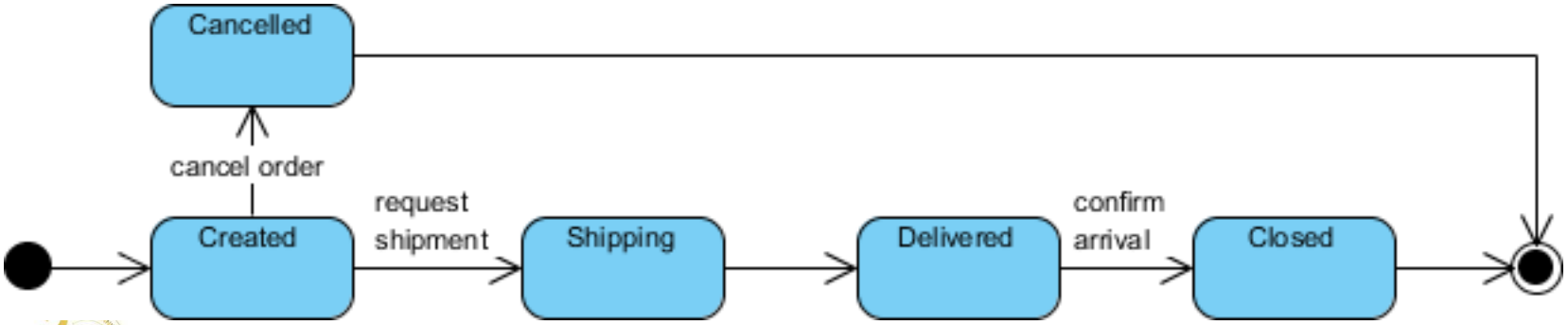
State

- A state represents a distinct condition or situation in the lifecycle of an object during which it satisfies some condition, performs some activity, or waits for an event.
  - Represents an abstraction or combination of values of its attributes or association
- Representation:



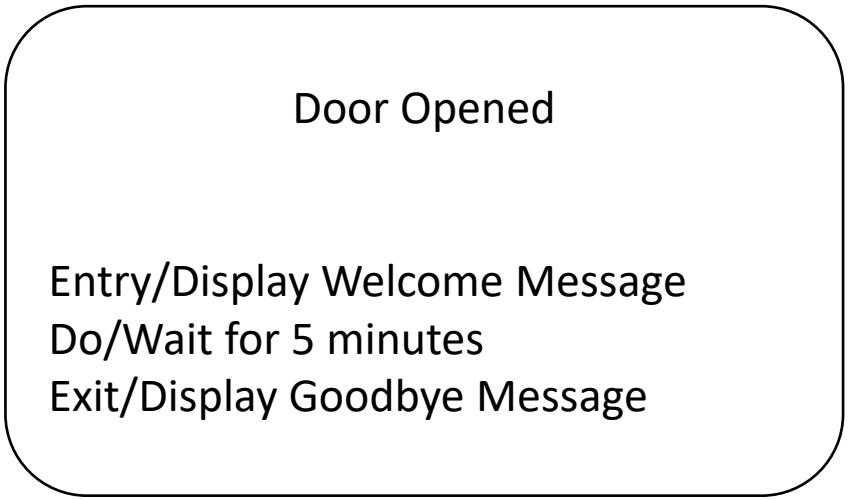
Trạng thái đầu Trạng thái trung gian Trạng thái kết thúc

- Properties: Finite, duration, stable, disjoin,complete



State Behavior

- An object can react to a specific event at a specific state. This is described in the state internal:
  - event/[action|activity]
    - Entry/action
    - Exit/action
    - Do/activity



Transition

- A state transition represents the movement of a system or object from one distinct state to another.
- A transition is instantaneous (takes no time), so it guarantees that the completeness of the state machine.
- The state transition happens at a specific condition: the trigger event
  - Triggerless transition: an automatic transition after the duration of the state is over.
- The guard condition
- The action

event(params) [condition]/action(params)



State Machine Diagram: Practices

- State machine diagram is often used during the business analysis to analyzing the lifecycle of a business entity
- State machine diagram can also be used in designing a specific object of the system (analysis object, design object)

ĐẠI HỌC BÁCH KHOA

25 YEARS ANNIVERSARY SOICT

VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

Chân thành cảm ơn!



soict.hust.edu.vn/ fb.com/groups/soict